

DATABASE + SERVER

From SQL to *an ORM*.

PostgreSQL stores the data. Express talks to it by hand. Sequelize learns to talk to it for you.

tables

SQL

pg

Express

Sequelize

PERSISTENCE

Apps need data that survives.

NOT ENOUGH

- ✗ Memory disappears on restart.
- ✗ Files are awkward for concurrent writes.
- ✗ Searching and filtering gets slow.
- ✗ Relationships become manual work.

DATABASE GIVES US

- ✓ Durable storage.
- ✓ Safe concurrent access.
- ✓ Fast querying.
- ✓ Data integrity with constraints.
- ✓ Transactions for all-or-nothing work.

the database is where important app state lives.

TWO PHILOSOPHIES

Strict shape, or flexible shape?

SQL (POSTGRESQL)

- ✓ Every row in a table follows the same schema.
- ✓ Columns and types are defined up front.
- ✓ The database rejects data that breaks the rules.
- ✓ Changing the shape means changing the schema.

NOSQL (E.G. MONGODB)

- ✓ Documents in a collection can differ in shape.
- ✓ No schema is enforced by default.
- ✓ Easy to add new fields on the fly.
- ✓ Your application code must keep data valid.

SQL: the database protects you. NoSQL: you protect yourself.

CORE CONCEPTS

Database words have exact meanings.

Term	Meaning
Database	A named collection of tables.
Table	Structured storage for one type of data.
Row	One record in a table.
Column	One field every row can have.
Schema	The definition: names, types, constraints.
Primary key	Unique identifier for each row, usually id.
Foreign key	A column that points at another table's primary key.

SCHEMA SHAPE

Types tell PostgreSQL what belongs.

SERIAL Auto-incrementing integer for ids.	INTEGER / FLOAT Whole numbers and decimals.	VARCHAR / TEXT Short bounded text or long text.
BOOLEAN True or false.	TIMESTAMP Date and time.	DATE Date only.

DDL

A table is storage with constraints.

```
CREATE TABLE users (  
  id          SERIAL PRIMARY KEY,  
  username    VARCHAR(50) NOT NULL UNIQUE,  
  email       VARCHAR(100) NOT NULL UNIQUE,  
  bio         TEXT,  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

CONSTRAINTS

- ✓ PRIMARY KEY uniquely identifies rows.
- ✓ NOT NULL requires a value.
- ✓ UNIQUE prevents duplicates.
- ✓ DEFAULT fills a value automatically.

INSERT / SELECT

Write rows. Read rows back.

```
INSERT INTO users (username, email, bio)
VALUES ('ada_lovelace', 'ada@example.com', 'First programmer');
```

```
SELECT * FROM users;

SELECT username, email
FROM users
WHERE id = 3
ORDER BY created_at DESC
LIMIT 10;
```

UPDATE / DELETE

The most dangerous missing word is **WHERE**.

CORRECT

```
UPDATE users
SET bio = 'Mathematician and writer'
WHERE id = 1;

DELETE FROM users
WHERE id = 4;
```

CATASTROPHE

```
UPDATE users
SET bio = 'oops';

DELETE FROM users;
```

without WHERE, every row is invited to the party.

FOREIGN KEYS

Tables can point at other tables.

```
CREATE TABLE posts (  
  id      SERIAL PRIMARY KEY,  
  title   VARCHAR(200) NOT NULL,  
  body    TEXT,  
  user_id INTEGER REFERENCES users(id) ON DELETE CASCADE  
);
```

`user_id` must match an existing `users.id`. Cascade means deleting the user deletes their posts too.

JOIN

JOIN puts related data back together.

```
SELECT posts.title, users.username
FROM posts
JOIN users
  ON posts.user_id = users.id;
```

JOIN TYPES

- ✓ JOIN returns rows with matches in both tables.
- ✓ LEFT JOIN keeps all rows from the left table.
- ✓ Unmatched right-side columns become NULL.

PSQL

The CLI is your database microscope.

```
psql -U postgres
psql -U postgres -d mydb
```

```
\l      -- list databases
\c mydb  -- connect
\dt      -- list tables
\d users -- describe table
\q       -- quit
```

SEED FILES

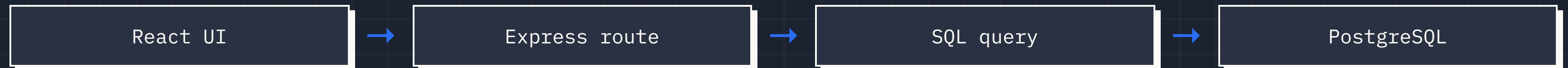
A .sql file can drop, create, and insert development data.

```
psql -U postgres -d mydb -f seed.sql
```

THE MISSING LINK

PostgreSQL doesn't speak JavaScript.

Everything so far happened inside psql. A real app needs Express to ask PostgreSQL for data and hand the answer to the browser.



Express is a client of PostgreSQL, the same way a browser is a client of Express.

THE PG DRIVER

pg sends strings, gets rows back.

```
npm install pg
```

```
// db/index.js
```

```
const { Pool } = require("pg")
```

```
const pool = new Pool({  
  connectionString: "postgres://localhost:5432/mydb"  
})
```

```
module.exports = pool
```

pg has no idea what a "user" is. It just runs the SQL you give it.

RAW SQL ROUTES

Every route writes its own SQL.

```
const router = require("express").Router()
const pool = require("../db")

router.get("/", async (req, res) => {
  const result = await pool.query("SELECT * FROM users")
  res.json(result.rows)
})

router.post("/", async (req, res) => {
  const { username, email } = req.body
  const result = await pool.query(
    "INSERT INTO users (username, email) VALUES ($1, $2) RETURNING *",
    [username, email]
  )
  res.status(201).json(result.rows[0])
})
```

the route shape never changes — req in, res out, exactly one response.

RAW SQL ROUTES

\$1, \$2 keep user input out of SQL.

```
router.patch("/:id", async (req, res) => {
  const { id } = req.params
  const { bio } = req.body
  const result = await pool.query(
    "UPDATE users SET bio = $1 WHERE id = $2 RETURNING *",
    [bio, id]
  )
  if (!result.rows[0]) return res.sendStatus(404)
  res.json(result.rows[0])
})

router.delete("/:id", async (req, res) => {
  await pool.query("DELETE FROM users WHERE id = $1", [req.params.id])
  res.sendStatus(204)
})
```

NEVER DO THIS

```
// SQL injection risk
`DELETE FROM users WHERE id = ${id}`
```

A user could send an `id` that is actually more SQL. Placeholders like `$1` stop that.

THE COST OF RAW SQL

Every table repeats the same boilerplate.

WHAT RAW PG ASKS OF YOU

- ✗ Write every SQL string by hand.
- ✗ Track placeholder order, \$1, \$2, ...
- ✗ Reach into `result.rows[0]` every time.
- ✗ Repeat all of this per table, per route.

WHAT IF A LIBRARY...

- ✓ wrote the SQL strings for you?
- ✓ handled placeholders automatically?
- ✓ handed back a plain JS object?
- ✓ still let you drop to raw SQL when needed?

that gap is exactly what Sequelize fills.

THE IDEA

An ORM maps rows to objects.

What you want	Raw pg	Sequelize
One user by id	<code>pool.query("SELECT...WHERE id=\$1", [3])</code>	<code>User.findByPk(3)</code>
Create a user	<code>pool.query("INSERT INTO...")</code>	<code>User.create({ username })</code>
Delete a user	<code>pool.query("DELETE...WHERE id=\$1")</code>	<code>user.destroy()</code>

same database, same SQL underneath. different interface on top.

INSTALL AND CONNECT

Sequelize still needs the pg driver.

RAW PG, FROM BEFORE

```
const { Pool } = require("pg")

const pool = new Pool({
  connectionString: "postgres://localhost:5432/mydb"
})
```

SEQUELIZE, TODAY

```
npm install sequelize pg pg-hstore

const { Sequelize } = require("sequelize")

const db = new Sequelize(
  "postgres://localhost:5432/mydb",
  { logging: false }
)
```

same connection string. Sequelize wraps pg, it doesn't replace it.

ONE MODEL, ONE TABLE

Models define database shape.

```
// models/User.js
const { DataTypes } = require("sequelize")
const db = require("../db")

const User = db.define("User", {
  username: { type: DataTypes.STRING, allowNull: false, unique: true },
  email: { type: DataTypes.STRING, allowNull: false, unique: true },
  bio: { type: DataTypes.TEXT }
})

module.exports = User
```

Sequelize adds id, createdAt, and updatedAt automatically.

TYPES BRIDGE

DataTypes map to PostgreSQL types.

STRING VARCHAR(255)	TEXT TEXT	INTEGER INTEGER
FLOAT FLOAT	BOOLEAN BOOLEAN	DATE / DATEONLY TIMESTAMP / DATE

DEVELOPMENT TABLE CREATION

db.sync() creates missing tables.

```
const db = require("./db")
require("./models/User")

db.sync()
  .then(() => {
    app.listen(8080)
  })
```

db.sync({ force: true }) drops and recreates tables. dev only.

SAME ROUTE, TWO WAYS

It's the same route. The data layer changed.

RAW PG

```
router.get("/", async (req, res) => {  
  const result = await pool.query(  
    "SELECT * FROM users"  
  )  
  res.json(result.rows)  
})
```

SEQUELIZE

```
router.get("/", async (req, res) => {  
  const users = await User.findAll()  
  res.json(users)  
})
```

req, res, and the response shape never moved. only how we asked PostgreSQL did.

CREATE AND READ

Every Sequelize query returns a Promise.

```
const user = await User.create({
  username: "ada_lovelace",
  email: "ada@example.com",
  bio: "First programmer"
})

console.log(user.id)
```

```
const users = await User.findAll()
const user = await User.findByPk(3)

const ada = await User.findOne({
  where: { username: "ada_lovelace" }
})
```

UPDATE AND DELETE

Find the row. Then change it.

```
// Pattern 1: instance update
const user = await User.findByPk(3)
if (!user) return res.sendStatus(404)

user.bio = "Updated bio"
await user.save()
```

```
// Delete instance
const user = await User.findByPk(3)
if (!user) return res.sendStatus(404)

await user.destroy()
res.sendStatus(204)
```

findByPk returns null, just like result.rows[0] could be undefined. check before touching it.

RELATIONSHIPS

Associations teach models how to relate.

```
// models/index.js
const User = require("./User")
const Post = require("./Post")

User.hasMany(Post, { foreignKey: "userId" })
Post.belongsTo(User, { foreignKey: "userId" })

module.exports = { User, Post }
```

this is the same foreign key idea from the SQL section, written as Sequelize calls instead of a REFERENCES clause.

EAGER LOADING

include loads related data.

```
// User with posts
const user = await User.findByPk(1, {
  include: Post
})

console.log(user.Posts)

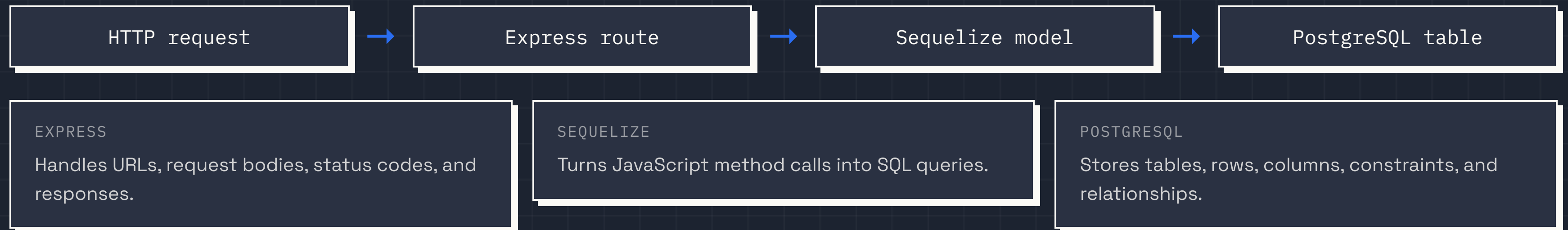
// Posts with authors
const posts = await Post.findAll({
  include: User
})
```

WHY IT MATTERS

- ✓ Associated data is not loaded by default.
- ✓ include performs eager loading.
- ✓ Prevents many separate follow-up queries.
- ✓ This avoids the N+1 problem.

BIG PICTURE

Know which layer you are touching.



POSTGRESQL COMMANDS

Shell commands versus psql commands.

IN YOUR TERMINAL

```
createdb ttp
dropdb ttp

psql -d ttp
psql -U postgres -d ttp
psql -d ttp -f seed.sql
psql -d ttp -c "SELECT * FROM users;"
```

INSIDE PSQL

```
\l          -- list databases
\c ttp       -- connect to database
\dt          -- list tables
\d users     -- describe one table
\q          -- quit
```

if it starts with a backslash, you are already inside psql.

SQL QUICK REFERENCE

The four CRUD commands in SQL.

```
INSERT INTO plants (name, type)
VALUES ('Pothos', 'Vine');
```

```
SELECT * FROM plants
WHERE type = 'Vine'
ORDER BY id DESC;
```

```
UPDATE plants SET watered = true
WHERE id = 3;
```

```
DELETE FROM plants
WHERE id = 3;
```

UPDATE and DELETE should make you look for WHERE immediately.

EXPRESS SETUP

A database-backed Express app has an order.

FILE SHAPE

```
app.js
database/
  db.js
  index.js
  plant.js
routes/
  plants.js
seed.js
```

STARTUP ORDER

- ✓ Import Express and middleware.
- ✓ Import db and models.
- ✓ Create app.
- ✓ Add `express.json()`.
- ✓ Mount routes.
- ✓ Add error middleware.
- ✓ `await db.sync()`, then `app.listen()`.

SEQUELIZE SETUP

Keep the database connection boring.

DATABASE/DB.JS

```
const { Sequelize } = require("sequelize")

const db = new Sequelize(
  "postgres://localhost:5432/ttp",
  { logging: false }
)

module.exports = db
```

DATABASE/INDEX.JS

```
const db = require("./db")
const Plant = require("./plant")

// associations go here later

module.exports = {
  db,
  Plant
}
```

MODEL PATTERN

A model is a table definition in JS.

```
const { DataTypes } = require("sequelize")
const db = require("./db")

const Plant = db.define("plant", {
  name: { type: DataTypes.STRING, allowNull: false },
  type: { type: DataTypes.STRING, allowNull: false },
  watered: { type: DataTypes.BOOLEAN, defaultValue: false }
})

module.exports = Plant
```

common options: type, allowNull, unique, defaultValue.

QUERY METHODS

Sequelize CRUD you will use daily.

Goal	Sequelize	Remember
Create	<code>Plant.create(req.body)</code>	Returns the new row.
Read all	<code>Plant.findAll()</code>	Returns an array.
Read one	<code>Plant.findByPk(id)</code>	Returns null if missing.
Filter	<code>findAll({ where: { type } })</code>	Like SQL WHERE.
Update	<code>plant.update(req.body)</code>	Find first, then update.
Delete	<code>plant.destroy()</code>	Find first, then destroy.

ROUTE PATTERN

Every async route needs the same bones.

```
app.get("/api/plants/:id", async (req, res, next) => {  
  try {  
    const plant = await Plant.findByPk(req.params.id)  
  
    if (!plant) {  
      return res.status(404).json({ msg: "not found" })  
    }  
  
    res.json(plant)  
  } catch (error) {  
    next(error)  
  }  
})
```

CHECKLIST

- ✓ Mark handler async.
- ✓ Use try/catch.
- ✓ await database calls.
- ✓ Check for null.
- ✓ Return after sending a 404.
- ✓ Call next(error) in catch.

ASSOCIATIONS + GOTCHAS

Relationships need setup before sync.

ASSOCIATION PATTERN

```
Plant.hasMany(CareNote, {
  foreignKey: "plantId"
})

CareNote.belongsTo(Plant, {
  foreignKey: "plantId"
})

await Plant.findPk(id, {
  include: CareNote
})
```

COMMON BUGS

- ✗ Forgetting await.
- ✗ Calling methods on null.
- ✗ Running db.sync() before associations.
- ✗ Using force: true in app startup.
- ✗ Forgetting express.json().
- ✗ Sending two responses from one route.

THE FULL STACK OF HABITS

Three layers, one set of instincts.

SQL

- ✗ Missing WHERE on update/delete.
- ✗ Confusing NULL with empty string.
- ✗ Assuming casing changes meaning.

RAW PG

- ✗ Building SQL with string concatenation.
- ✗ Losing track of \$1, \$2 order.
- ✗ Forgetting to check `result.rows[0]`.

SEQUELIZE

- ✗ Forgetting `await`.
- ✗ Using `force: true` outside dev.
- ✗ Calling `include` before associations exist.

whichever layer you're in: check before you touch it, and never trust raw input in a query.